

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

In parallelo

Più subagent, un messaggio

Lavori indipendenti, insieme; poi una sintesi

- Un **subagent** lavora in una finestra di contesto sua, e alla conversazione principale torna solo il suo riassunto. Qui, **più lavori insieme**
- Per indagini **indipendenti**, più subagent lavorano **in contemporanea**: ognuno esplora la sua parte, poi Claude **sintetizza**. Funziona meglio quando i percorsi non dipendono l'uno dall'altro. L'esempio della documentazione: «Research the authentication, database, and API modules in parallel using separate subagents»
- Finiscono nel tempo del **più lento**, non nella somma dei tempi. Si avviano con più chiamate al tool **Agent** (prima si chiamava Task) nella **stessa risposta**, non in turni separati
- Due schemi di Anthropic: il *sectioning*, sottocompiti indipendenti in parallelo, e il *voting*, lo stesso compito eseguito più volte. E un agente che **coordina**: ogni subagent vuole un **obiettivo**, un **formato** di risposta, indicazioni su strumenti e fonti, e **confini** chiari

Stessi file? Un worktree per filone

Due sessioni sugli stessi file si scontrano: un checkout a testa

- I compiti toccano gli **stessi file**? Due strade: **una sola sessione**, un compito dopo l'altro; oppure un **worktree** per filone, un checkout a sé su un ramo suo
- `claude -w <nome>`, o `--worktree`: un worktree **isolato**, su un ramo nuovo `worktree-<nome>`, e Claude parte lì

```
# in un repository git, con almeno un commit
claude -w esempio     # un terminale: il ramo worktree-esempio
claude -w rinomina    # un altro: il ramo worktree-rinomina
git worktree list     # il checkout principale e i due worktree
```

- Da lì Claude Code **blocca** Edit, Write e NotebookEdit diretti al checkout principale; **non** le scritture della shell, come un `cp` o una redirezione
- Serve **almeno un commit**: un worktree parte da un commit esistente; senza un remote, dall'HEAD locale
- Con `claude -p` non c'è la domanda all'uscita: i worktree **restano**, e si tolgono a mano
- Anche un subagent può avere il suo: `isolation: worktree` nel frontmatter; se non cambia nulla, il worktree si toglie da solo

M4-D07 Due filoni, due worktree

Demo del docente in Claude Code: due sessioni insieme, senza e con i worktree

> Dalla pagina «Run parallel sessions with worktrees»: «Pass --worktree or -w with a name to create an isolated worktree and start Claude in it.…on a new branch named worktree-<name>». La coppia la **mette alla prova**

> E: «File edits: Claude Code blocks an Edit, Write, or NotebookEdit that targets a path in the main checkout.» Questa la coppia **non** la mette alla prova: il controllo la decide solo se una sessione prova a modificare il checkout principale

`prompt-esempio.txt, il testo di una delle due sessioni, a capo per la slide`

```
<instructions>
Aggiungi alla docstring del modulo progetti/urgenze/urgenza.py
un esempio d'uso della funzione urgente, nel formato di
doctest: righe che cominciano con >>>.
</instructions>

<constraints>
Lascia invariato tutto il resto.
</constraints>
```

M4-D07 Due filoni, due worktree

Demo del docente in Claude Code: due sessioni insieme, senza e con i worktree

⚖ Due modifiche allo **stesso file**, `progetti/urgenze/urgenza.py`, chieste da due sessioni partite **insieme**. Due bracci che differiscono in una cosa sola: le due sessioni nel **checkout principale**, oppure ciascuna nel **suo worktree**

`prompt-rinomina.txt, il testo dell'altra sessione, a capo per la slide`

```
<instructions>
Rinomina il parametro giorni_aperta della funzione urgente in
giorni_di_apertura, in progetti/urgenze/urgenza.py e nei suoi
test, progetti/urgenze/test_urgenza.py.
</instructions>

<constraints>
Lascia invariato tutto il resto.
</constraints>
```

M4-D07 Due filoni, due worktree

Demo del docente in Claude Code: due sessioni insieme, senza e con i worktree

- **Il comando**, nello spazio di lavoro di M4-D07 (README, passo 3): le due sessioni partono insieme, & e poi `wait`, e `fotografa.sh` prende subito lo stato di git del braccio. Prima il braccio senza worktree, poi `git checkout -- progetti/urgenze`, poi quello con `-w`

```
e="$(cat prompt-esempio.txt)"
r="$(cat prompt-rinomina.txt)"
o=(--model sonnet --effort low --allowedTools Read,Edit
   --strict-mcp-config --no-session-persistence
   --output-format stream-json --verbose)
# [...] il braccio senza worktree: le stesse righe, senza -w
claude -p "$e" -w esempio "${o[@]}" \
  > esecuzioni/con-worktree/esempio.jsonl &
claude -p "$r" -w rinomina "${o[@]}" \
  > esecuzioni/con-worktree/rinomina.jsonl &
wait
bash fotografa.sh con-worktree
```

- Solo **Read** ed **Edit** hanno il permesso: in `claude -p` le chiamate che chiederebbero un'approvazione sono **negate**. Così una modifica passa solo da Edit, che il blocco copre, e non da una scrittura della shell, che il blocco non copre
- Da guardare: senza worktree, `git diff --stat`; con i worktree, `git branch --list 'worktree-*'`, un ramo per sessione; poi la verifica

M4-D07 Due filoni, due worktree

Demo del docente in Claude Code: due sessioni insieme, senza e con i worktree

La registrazione del corso, effort low: il controllo sugli stream e sulle fotografie di git

sessione	dove è partita	il suo `urgenza.py`
senza worktree: esempio e rinomina	tutte e due nel checkout principale	l'esempio sì, la rinomina sì: le due modifiche nello stesso file, nello stesso checkout
con `-w`: esempio	`.claude/worktrees/esempio`, ramo `worktree-esempio`	l'esempio sì, la rinomina no
con `-w`: rinomina	`.claude/worktrees/rinomina`, ramo `worktree-rinomina`	l'esempio no, la rinomina sì

Un braccio dopo l'altro, le due sessioni insieme: i valori che il controllo ha stampato, in tabella
Registrazione del corso, 1 novembre 2026, Claude Code 2.1.316, Sonnet

- «Con -w, un worktree isolato su un ramo nuovo, e Claude parte lì» e «Ogni worktree tiene soltanto la modifica della sua sessione»: **confermata**, per l'una e per l'altra sessione. Con i worktree, il checkout principale: **nessuna modifica** ai file tracciati
- «Le modifiche al checkout principale sono bloccate»: **non distinguibile** — nessuna sessione ha provato a modificare il checkout principale, e il blocco di Edit, Write e NotebookEdit **non è stato messo alla prova**
- Dopo `claude -p` i worktree c'erano **ancora**: la pulizia, per ognuno, `git worktree remove --force --force` e `git branch -D`

In background

Mentre un lavoro gira, Claude continua; e una schermata per tutte le sessioni

- Un comando lungo può girare **in background**: Claude lo avvia con `run_in_background`, e intanto continua a lavorare. **Ctrl+B** manda in background i comandi Bash e gli agenti in esecuzione
- `/tasks` elenca ciò che gira in background nella sessione: lo controllate, vi ci collegate, lo fermate
- Il comando continua anche **dopo la risposta finale**; un `claude -p` resta aperto finché il comando non finisce o non raggiunge il suo **limite di tempo**
- **Agent view**: `claude agents` mostra in una schermata tutte le sessioni in background — che cosa gira, che cosa aspetta voi, che cosa è finito. `claude --bg` ne avvia una; `attach`, `stop` e `rm` la gestiscono
- Agent view è in **research preview**: interfaccia e scorciatoie possono cambiare

Agent teams e workflow

Due modi di Claude Code che il corso nomina soltanto

- **Agent teams**: una sessione fa da **team lead**, assegna i compiti e sintetizza; i compagni si parlano direttamente. Sono **sperimentali**, spenti per impostazione predefinita; in `claude -p` non partono; con i compagni in plan mode usano circa **7 volte** i token di una sessione normale
- Due compagni sullo **stesso file** si **sovrascrivono**: il lavoro va diviso perché ognuno abbia i suoi file. Per modifiche allo stesso file, meglio una sessione sola o i subagent
- Nel compilatore C di Anthropic (febbraio 2026), **16 agenti** in quasi 2.000 sessioni di Claude Code: a un certo punto ognuno trovava lo stesso bug, lo correggeva e **sovrascriveva** le modifiche degli altri
- **Dynamic workflows**: uno script JavaScript che orchestra molti subagent. Sul Pro si accendono dalla riga «Dynamic workflows» di `/config`, con la dimensione **small**
- Nel corso li nominiamo soltanto: **nessuna esercitazione** li usa

Quanto regge il Pro

Il lavoro in parallelo si paga in token: si comincia con due o tre filoni

- Più sessioni o subagent insieme **moltiplicano** i token. Le sessioni in background consumano l'abbonamento come quelle interattive: **dieci** agenti in parallelo consumano la quota circa **dieci volte** più in fretta
- Nel post di Anthropic sul suo sistema di ricerca multi-agente (giugno 2025): un agente usa di solito circa **4 volte** i token di una chat, un sistema multi-agente circa **15 volte**
- Per cominciare, **due o tre** sessioni: il limite pratico è quante una persona riesce a **rivederne bene**
- E la maggior parte dei compiti di programmazione ha meno parti **davvero parallele** della ricerca
- A voce: quale lavoro della vostra settimana si divide in parti indipendenti?

In parallelo solo ciò che è indipendente: un worktree per filone, e il conto cresce con i filoni.